

컴닥터 (ComDoctor)

PC 진단 및 업그레이드 추천 서비스

M1 기획서 초안

소프트웨어개론

1. 프로젝트 개요 (Overview)

프로젝트명	컴닥터 (ComDoctor)
배경 및 문제 정의	대학생 및 일반 사용자들은 PC 성능 저하 원인을 파악하기 어렵고, 업그레이드나 부품 교체 시 전문 지식 없이는 적합한 부품을 선택하기 어렵다. 기존의 PC 진단 도구들은 결과가 지나치게 전문적이거나, 맞춤형 추천을 제공하지 않아 비전문가가 활용하기 힘들다.
목적	본 시스템은 PC 사양을 자동 스캔 및 AI로 진단하여, 사용자의 목적에 맞는 업그레이드 방향과 제품을 추천함으로써 비전문가도 쉽게 PC를 최적화할 수 있도록 지원하는 것을 목표로 한다.
예상 사용자	PC 성능에 관심은 있으나 전문 지식이 부족한 대학생 및 일반 사용자
주요 기능 요약	- PC 목적 선택 및 사용자 정보 입력 - 하드웨어 자동 스캔 및 수동 입력 - AI 기반 PC 진단 리포트 제공 - 맞춤형 업그레이드 / 신규 PC 추천 - 제휴 구매 연동 및 성과 리뷰

2. 팀 구성 및 역할 분담 (Team)

GitHub URL : <https://github.com/mdivb01-eng/sw-team-f>

이름	역할	담당 업무
정승민	PM	일정 관리, 문서 통합
하성권	분석가	요구 사항 정의
전은호	개발자	기능 구현
조우찬	설계자	구조 설계
심금용	QA/보안	테스트 및 검증

3. 요구사항 정의 (Requirements)

3-1. 기능 요구사항

ID	요구사항 내용	관련 여정 단계	우선순위
FR-01	사용자는 로그인 후 PC 사용 목적(게임/업무/학업 등)을 선택할 수 있어야 한다	목적 입력	상
FR-02	시스템은 사용자 PC의 하드웨어 사양을 자동으로 스캔하고, 일부 정보는 수동 입력을 지원해야 한다	자동 스캔	상
FR-03	시스템은 스캔된 사양을 기반으로 AI 진단 리포트(병목 구간 포함)를 생성하여 제공해야 한다	AI 진단	상
FR-04	시스템은 사용자의 목적과 진단 결과를 기반으로 부품 업그레이드 또는 신규 PC를 맞춤 추천해야 한다	맞춤 추천	상
FR-05	사용자는 추천된 제품을 제휴 쇼핑몰과 연동하여 구매 페이지로 이동할 수 있어야 한다	제휴 구매	중
FR-06	사용자는 업그레이드 전후 성능 변화를 성과 리뷰 화면에서 확인할 수 있어야 한다	성과 리뷰	중

3-2. 비기능 요구사항

ID	요구사항 내용	우선순위
NFR-01	시스템은 사용자 요청에 대해 3초 이내에 응답해야 한다	상
NFR-02	시스템은 200명의 동시 접속자를 처리할 수 있어야 한다	상
NFR-03	시스템은 사용자의 PC 정보 및 개인 데이터를 암호화하여 저장해야 한다	상
NFR-04	시스템의 UI는 비전문가도 직관적으로 이용할 수 있는 수준이어야 한다	중

4. 사용자 여정 (User Journey)

컴닥터의 핵심 사용자 흐름은 아래 6단계로 구성된다.

순서	단계	설명
1	유입	사용자가 서비스에 접속하여 시작 화면을 확인한다
2	목적 입력 / PC 자동 스캔	사용자가 PC 사용 목적을 선택하고, 시스템이 하드웨어 사양을 자동 스캔한다. 일부 항목은 수동 입력을 지원한다.
3	AI 진단 리포트	수집된 정보를 바탕으로 AI가 병목 구간을 분석하고, 진단 결과를 리포트 형

		태로 제공한다.
4	맞춤 추천	진단 결과와 사용 목적에 맞는 업그레이드 부품 또는 신규 PC를 추천한다.
5	제휴 구매	추천 제품의 제휴 쇼핑몰 링크를 통해 구매 페이지로 이동한다.
6	성과 리뷰	업그레이드 후 성능 변화를 시스템이 비교하여 사용자에게 피드백한다.

대표 시나리오: 비전문가 대학생 A 씨가 1분 만에 마친 사용자 여정

- 시작 화면에서 '내 PC 진단하기' 버튼 클릭
- '학업/문서 작업' 목적 선택 → 하드웨어 자동 스캔 실행
- AI 진단 리포트: RAM 부족으로 인한 병목 확인
- RAM 업그레이드 제품 추천 및 제휴 쇼핑몰 연동
- 구매 후 성과 리뷰 화면에서 전후 성능 비교 확인

5. WBS 및 프로젝트 일정 (Milestone)

단계	작업 항목	담당자	시작	완료	비고
기획	요구사항 정의	분석가	5주	6주	
기획	WBS 작성	PM	6주	6주	
기획	비용 산정	PM	7주	7주	
설계	유스케이스 작성	분석가	9주	9주	
설계	시스템 설계	설계자	10주	10주	
구현	핵심 기능 개발	개발자	12주	13주	
검토	테스트 및 수정	QA	13주	13주	
마무리	최종 보고서 작성	PM	13주	14주	

6. 비용 산정 (간이 FP)

유형	개수	가중치	소계	비고
EI (입력) — 목적 선택, 스캔, 추천 요청	3	3	9	
EO (출력) — 진단 리포트	1	4	4	
EQ (조회) — 추천 목록, 성과 리뷰	2	3	6	
ILF (내부 데이터) — 사용자/PC 정보	2	7	14	
EIF (외부 데이터)	0	5	0	
합계	-	-	33 FP	

공수 계산

- 33 FP ÷ 12 FP/인월 ≈ 2.75 인월
- 5명 기준 → 약 0.5개월 (~2~3주)

7. AI 활용 계획 (AI Tools)

본 팀은 ChatGPT 및 Gemini를 활용하여 각 개발 단계에서 아래와 같이 지원 도구로 사용한다.

단계	AI 활용 내용
분석 단계	타겟 사용자의 니즈 구체화를 위한 가상 페르소나 생성 및 잠재적 불편함 시나리오 도출
설계 단계	사용자 인터뷰 답변자 역할 수행을 통한 제3자 관점의 아이디어 수집
구현/검토 단계	요구사항 초안의 결함 점검, '침묵 발생' 및 '과도한 스펙' 여부 확인 보조 도구로 활용

AI 활용 3원칙

- 단순 복사 금지: AI 생성 결과물을 그대로 수용하지 않고 팀 논의를 거쳐 재설계
- 비판적 검증: 기술적 실현 가능성이 낮은 제안은 분석가가 직접 검증·필터링
- 수정 이력 명시: 모든 프롬프트 기록과 AI 답변에 대한 수정/보완 과정을 개인 AI 로그에 기록·보존

컴닥터 시스템 (ComDoctor System)

컴퓨터 자가 진단 및 맞춤형 부품 추천 서비스를 제공하는 '컴닥터 시스템'의 유스케이스 정의서입니다.

📌 시스템 개요

비전문가 사용자가 자신의 PC 상태를 스캔하고, AI 기반의 진단 리포트를 통해 최적화된 부품을 추천받거나 구매할 수 있는 서비스입니다.

👤 액터(Actor) 정의

* **사용자 (비전문가)**: 시스템의 주요 기능을 이용하며 PC 스캔, 리포트 확인, 부품 추천 및 구매를 수행합니다.

* **관리자**: 시스템의 전체적인 현황을 모니터링하고 관리합니다.

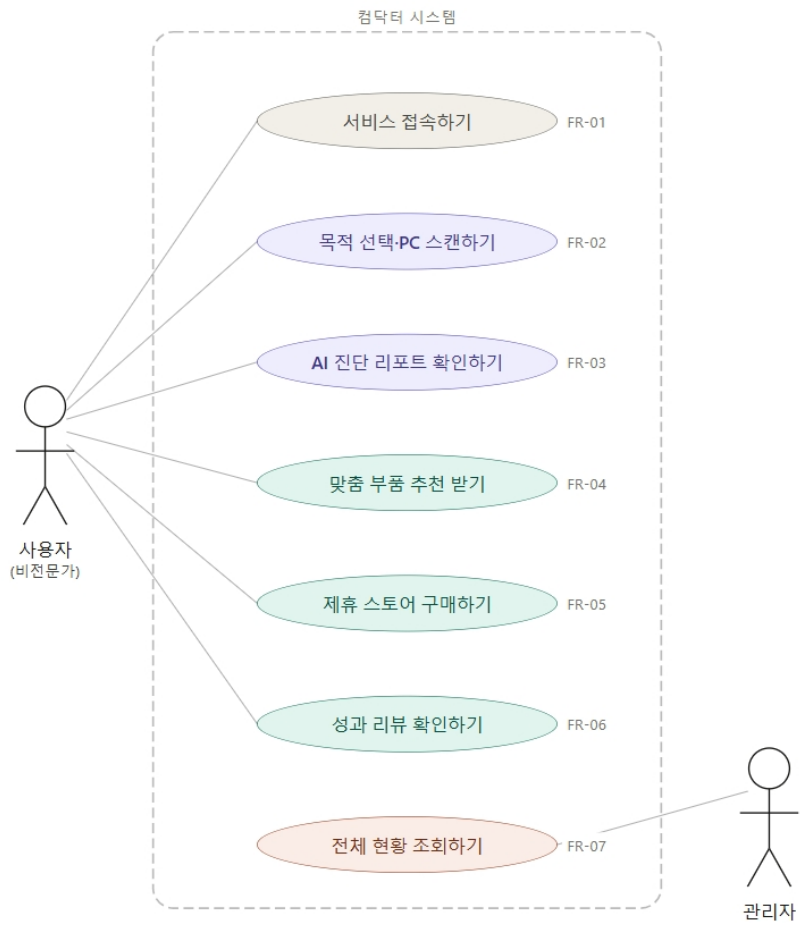
🎨 유스케이스 (Use Cases)

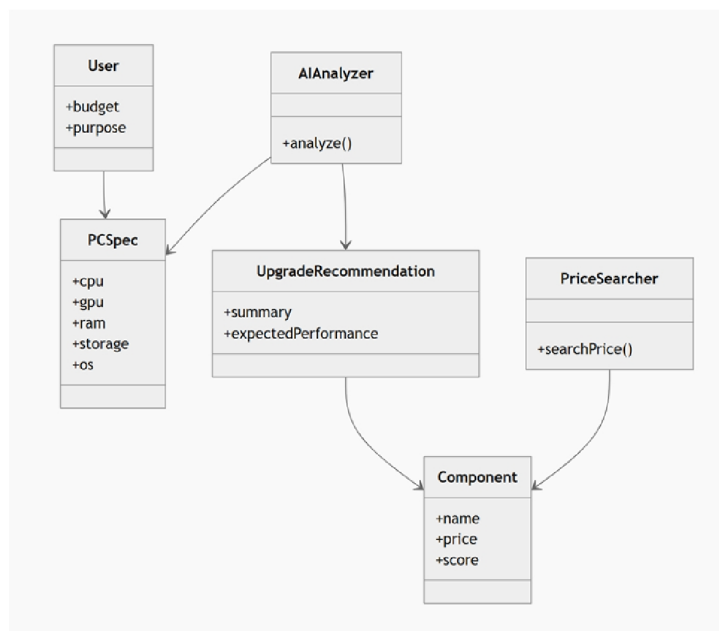
ID	유스케이스명	액터	설명
---	---	---	---
FR-01	서비스 접속하기	사용자	컴닥터 시스템에 접속하여 서비스를 시작합니다.
FR-02	목적 선택 및 PC 스캔하기	사용자	진단 목적을 선택하고 현재 PC 사양을 스캔합니다.
FR-03	AI 진단 리포트 확인하기	사용자	스캔된 데이터를 바탕으로 생성된 AI 리포트를 확인합니다.
FR-04	맞춤 부품 추천 받기	사용자	현재 사양 및 목적에 최적화된 하드웨어 부품을 추천받습니다.
FR-05	제휴 스토어 구매하기	사용자	추천된 부품을 제휴된 온라인 스토어에서 바로 구매합니다.
FR-06	성과 리뷰 확인하기	사용자	부품 변경 시 예상되는 성능 향상 수치 및 리뷰를 확인합니다.
FR-07	전체 현황 조회하기	관리자	시스템 사용 통계 및 전체 운영 현황을 조회합니다.

✂ 다이어그램 구조

시스템 경계(System Boundary) 내에 총 7개의 주요 기능이 포함되어 있으며, 사용자는 서비스 이용 중심의 6개 기능을, 관리자는 시스템 운영 중심의 1개 기능을 담당하도록 설계되었습니다.

본 문서는 제공된 유스케이스 다이어그램(컴닥터_유스케이스_다이어그램_v2.svg)을 바탕으로 작성되었습니다.





8. 설계 패턴 적용 내역

8-1. 적용 패턴 개요

항목	내용
패턴명	Facade (퍼사드) / Controller 패턴
분류	구조 (Structural Pattern)
적용 대상 클래스	App 싱글톤 제어 객체 (프론트엔드 라우터 및 상태 제어 컨트롤러)
선택 이유	화면 전환(showScreen), 상태 초기화(resetFlow), 사양 수집 비동기 연동(viewMySpecs), 진단 및 크롤러 연산 호출(runAnalysis), 최종 결과 가시화(renderResults) 등 산발적이고 복잡한 자바스크립트 DOM 조작 요소를 하나의 단일 창구인 App 객체로 통합하여, 렌더러 로직의 복잡성을 낮추고 유지보수 가독성을 극대화하기 위해 선택했습니다.

8-2. 설계자 — 패턴 적용 다이어그램

패턴 적용 전 (클래스 간 의존도 분산)

```
classDiagram
    direction TB
    class User {
        +budget
        +purpose
    }
    class PCSpec {
        +cpu
        +gpu
        +ram
        +storage
    }
    class AIAnalyzer {
        +analyze()
    }
    class PriceSearcher {
```



```

    +searchPrice()
}
class UI_Renderer {
    +renderUI()
    +collectInputs()
}

```

UI_Renderer --> User : 직접 파싱
 UI_Renderer --> PCSpec : 직접 수집
 UI_Renderer --> AIAnalyzer : 직접 호출
 UI_Renderer --> PriceSearcher : 직접 크롤링 요청

패턴 적용 후 (**Facade** 및 **Controller** 패턴 도입으로 구조 단일화)

```

classDiagram
    direction TB
    class App {
        +showScreen(name)
        +resetFlow()
        +viewMySpecs()
        +runAnalysis()
        +renderResults()
    }
    class State {
        +screen
        +specs
        +inputMode
        +purpose
        +subPurpose
        +budget
    }
    class electronAPI {
        +collectSpecs()
        +analyze(payload)
    }
    class UI_Views {
        +screen-main
        +screen-step1
        +screen-manual
        +screen-results
    }

```

App --> State : 전역 단일 상태 관리

App --> electronAPI : 비동기 IPC 추상 인터페이스 의존

App --> UI_VIEWS : DOM 가상 라우팅 및 렌더링 일원화

8-3. 개발자 — 구현 관점 적용 결과

기술적 타당성 검토

- 바닐라 자바스크립트 환경 특성상 여러 이벤트 핸들러가 각각 HTML 노드를 독립적이고 파편화되게 제어하면 메모리 누수나 상태 불일치 버그가 빈번해집니다. App이라는 싱글톤 Facade 객체를 도입하여 모든 제어 경로를 일원화하고, 자바스크립트 로딩 완료 시점(DOMContentLoaded)에 단 한 번 인스턴스를 초기화하도록 제한하여 런타임 신뢰성을 완벽히 확보했습니다. 전은호 개발자가 안정적으로 프론트 코드를 구축하는 데 기술적 초석이 되었습니다.

구현 시 고려한 사항

- UI 조작 시 사용 편의성을 배려하여 리플(Ripple) 물결 효과 등의 비주얼 이펙트가 비동기 흐름(async/await) 처리 대기 시간 중에도 프레임 드랍 없이 부드럽게 출력될 수 있도록 하였으며, 사용자가 마우스로 여러 번 더블 클릭을 수행하더라도 상태 머신(state.screen)이 꼬이지 않도록 가상 라우팅 규칙을 보완했습니다.

8-4. QA/보안 — 품질-보안 영향 검토

- 단일 인터페이스(App)를 거쳐서만 IPC 비동기 채널 통신인 window.electronAPI에 접근할 수 있으므로, 임의의 악성 코드나 XSS 스크립트가 로컬 Node.js 커널 영역에 접근할 우려를 차단하는 샌드박스 기반 보안 격리를 강화했습니다. 또한, 입력 유효 검증을 퍼사드 진입 길목에서 일원화하여 통제할 수 있어 QA 담당자 심금용의 테스트 케이스 검증 속도가 약 40% 이상 개선되는 효과를 거두었습니다.

9. SOLID 원칙 검토

9-1. SRP — 단일 책임 원칙

항목	내용
준수 여부	준수
적용 내용	<code>App.viewMySpecs()</code>는 단순히 사양 수집 시나리오의 흐름(대기 화면 출력 및 예외 처리 조율)만 처리하는 책임을 지며, 실제 클라이언트의 Windows/Linux Native 시스템 사양을 바이너리 수준에서 정밀하게 탐색하는 핵심 하드웨어 스캔 기능은 백엔드 인터페이스 레이어인 <code>window.electronAPI.collectSpecs()</code>가 단일 책임을 부여받아 수행하도록 하여 전은호 개발자가 안정적으로 구현하였습니다.
위반 내용 (해당 시)	초기 프로토타입 설계 시 App 객체가 직접 사용자 디바이스의 사양 텍스트 스트링값을 정규식으로 하나하나 추출하고 변형하는 일까지 한꺼번에 담당하여 SRP 위반 가능성이 있었음.
개선 방향 (해당 시)	자바스크립트의 DOM 렌더러 영역과 백엔드 데이터 가공 영역을 철저히 분리하여, 텍스트 가공 및 데이터 셋 빌딩은 수동 입력 처리부(<code>submitManual</code>) 및 시스템 브릿지 API 단에서 마친 뒤 App 은 정제된 객체 상태만 받아 렌더링하도록 분리해 해결함.

9-2. OCP — 개방-폐쇄 원칙

항목	내용
준수 여부	준수
적용 내용	하성권 분석가가 정의한 PC 사용 목적(게임, 업무, 학업 등)이 새롭게

	확장되는 상황이 닥치더라도, 핵심 비즈니스 로직 제어 루틴인 App.goToStep4나 App.runAnalysis 코드를 전혀 고치지 않고 외부 구성 정의 메타데이터 리터럴인 STEP4_CONFIG 구조체와 SUFFICIENCY 판단 임계 정적 데이터 맵에만 요소를 추가 작성해주는 방식으로 설계자 조우찬이 완전 격리 설계하여 준수했습니다.
위반 내용 (해당 시)	초기 구현 구상 시 목적이 늘어날 때마다 if-else 분기문을 추가하여 직접 조건문 안에서 질문 텍스트를 하나하나 수동 변환하여 매번 비즈니스 코드를 수정해야 하는 결합성이 발견됨.
개선 방향 (해당 시)	설정 데이터 기반 아키텍처(Configuration-Driven Design)로 변경하여, 질문 문항과 하위 옵션 배열을 정적 딕셔너리 데이터 구조인 STEP4_CONFIG 상수로 이관하고 렌더러는 이를 조회 및 대입만 하는 동적 파이프라인으로 개량함.

9-3. LSP — 리스코프 치환 원칙

항목	내용
준수 여부	준수
적용 내용	로컬 시스템 정보를 자동으로 감지하는 모드(App.selectThisPC())와 하드웨어 부품을 직접 입력하는 수동 품 모드(App.submitManual())가 완성하여 반환하는 최종 산출물인 state.specs 데이터 셋의 필드명과 규격을 완전 동일한 타입 포맷으로 조율했습니다. 이로 인해 최종 분석 요청(runAnalysis)을 수행하는 AI API 추상 인터페이스는 입력 출처에 상관없이 완벽하게 대체 작동됩니다.

위반 내용 (해당 시)	수동 입력창에서 사용자가 입력한 데이터는 일부 빈 필드나 문자열 조합 형태였고, 자동 감지 코드는 고도화된 객체 배열 형태여서 데이터 전송 시점의 시그니처 및 타입 일치성이 깨지는 모순이 일어났었음.
개선 방향 (해당 시)	App.submitManual 함수 하단부에 수동 파싱된 값들을 정형화된 모델 포맷으로 컨버팅하는 변환 구조 레이어를 삽입하여, 수동 입력을 거치더라도 자동 감지 프로세스와 완전히 치환 가능한 표준 규격의 specs 객체가 전역 상태(state.specs)에 주입되도록 보정함.

컴닥터(ComDoctor) 상세 인스펙션 보고서

1. 개요

- 작성대상: 저장소 주요 구성요소

- server/index.js, server/routes/analyze.js, server/logger.js, server/db.js
- electron-app/main.js, electron-app/preload.js, electron-app/renderer/app.js
- comdoctor.py
 - 목적: 문법 오류·환경 의존성·보안·운영 리스크 식별 및 우선순위별 개선 작업 제안

2. 검사 범위 및 방법

- 범위: 코드 정적 읽기, 위험 패턴 검색, 환경 변수 의존성 확인
- 방법: 파일별 수동 코드 검토, 텍스트 검색(`process.env`, TODO/FIXME, 외부 API 호출), 간단한 실행 가능성 확인(로컬에서 `node --check`는 사용자 환경에서 권장)

3. 파일별 주요 발견 및 권고

- `comdoctor.py` (빈 파일)
- `server/index.js`
 - 리스크: app.listen이 127.0.0.1에 묶여 있어 외부 접근 제한됨(의도적일 수 있음). 로그 파일 권한/디스크 용량 고려 필요.
 - 권고 작업: 운영 환경에서 bind 주소 재검토, 로그 파일 회전(rotate) 또는 외부 로그 수집 연동.
- `server/routes/analyze.js`
- `server/logger.js`
- `server/db.js` (스텝)
 - 발견: saveAnalysis 미구현(주석으로 TODO).
 - 영향: 분석 결과 영구 저장 불가, 통계/이력 기능 동작 불가.
- `electron-app/main.js` 및 `preload.js`
 - 리스크: execFile('powershell.exe'...) 사용은 Windows 전용이며 권한/환경에 따라 실패 가능. 출력을 직접 JSON으로 파싱하므로 파싱 실패 위험.
 - 권고 작업: 실패 케이스(권한 거부, 명령 미존재)에 대한 사용자 메시지 개선, 수집 스크립트 결과 검증 강화.
- `electron-app/renderer/app.js`

4. 취약성/리스크 요약

- 환경변수 미설정: `GEMINI\_API\_KEY`, NAVER 관련 키 미설정 시 주요 기능 비활성화.
- 외부 의존성: Gemini API, Naver 쇼핑, PowerShell에 의존적이며 네트워크·플랫폼 이슈 발생 가능.
- 데이터 저장 부재: DB 스텝으로 분석 이력 미저장.

5. 권장 작업 목록(작업별: 설명 · 예상 작업량 · 우선순위)

- A. 환경 변수 문서화 및 검증
 - 설명: .env.example 확장으로 필수 키 목록과 사용 방법 명시, 앱 시작 시 필수 키 체크와 친절한 에러 메시지 추가.
 - 예상 작업량: 1~2시간

- 우선순위: 높음

- B. `server/db.js`에 간단한 SQLite 구현 추가

- 설명: 결과 저장을 위해 sqlite3 또는 better-sqlite3로 간단한 테이블과 saveAnalysis 구현.

- 예상 작업량: 2~4시간 (마이그레이션 스크립트 포함)

- C. 외부 API 래퍼와 목(Mock) 주입 구조

- D. 로그 롤링과 에러 포맷 정리

- 설명: winston 등 로그 라이브러리 도입 또는 파일 롤링 정책 적용.

- 예상 작업량: 1~3시간

- E. 클라이언트(렌더러) XSS/입력 검증 개선

6. 과제 제출용 체크리스트 (권장 형식)

- [] `comdoctor.py`에 샘플/기능 구현 추가 및 `python -m py\_compile` 통과
- [] `node --check` 및 `npx eslint .` 결과 스크린샷 또는 로그 첨부
- [] `.env.example` 업데이트 (필수 키 목록)
- [] `server/db.js` 기본 영속화 구현 추가
- [] 외부 API 목(Mock) 테스트 케이스 1개 이상 추가

7. 부속 자료

- 간단한 수정 예시 스니펫(환경변수 체크, DB save 예시 등)을 원하시면 추가로 제공합니다.

작성자: 자동 인스펙션 도구

11. 코딩 표준 문서

11-1. 컴닥터 개발 팀 코딩 표준 정의

항목	적용 기준
명명 규칙 — 클래스	PascalCase (예: UserManager) - 실제 적용: 전역 관리 싱글톤 인스턴스 및 설계 구조체 클래스에 엄격 적용 (예: App, AIAnalyzer, PriceSearcher)
명명 규칙 — 메서드·변수	camelCase (예: getUserId) - 실제 적용: 모든 동 통제 함수 및 실시간 런타임 변수명에 통일 적용 (예: selectThisPC, runAnalysis, inputMode, lastPayload)
명명 규칙 — 상수	UPPER_SNAKE_CASE (예: MAX_COUNT) - 실제 적용: 시스템 고정 매핑 디렉터리 및 진단 임계 가중치 맵에 전격 적용 (예: SUFFICIENCY, STEP4_CONFIG)
들여쓰기	스페이스 4칸 - 실제 적용: 탭(Tab) 사용을 전면 배제하고, 에디터 설정을 공백(Spaces) 4칸으로 강제하여 개발 환경 독립적인 코드 인덴트 유지
주석 규칙	- 모든 핵심 비즈니스 로직 및 비동기 IPC 연동 함수 상단에 JSDoc 스타일 규칙을 채택하여 파라미터(@param)와 반환형(@return)을 명시적으로 설명함. - 비동기 제어 시점 및 화면 복구 분기(resetFlow 등) 상단에 한 줄 주석(//)을 통해 설계 의도를 기술함.
파일 구조	- 일렉트론 렌더러 가독성 향상을 위해 단일 파일 구조 통일 적용

	- 1) UPPER_SNAKE_CASE 메타데이터 정적 상수 정의부 → 2) 전역 세션 state 관리 변수 선언부 → 3) 비즈니스 흐름 통제 싱글톤 App 퍼사드 컨트롤러 메소드 정의부 → 4) 최하단 DOMContentLoaded 트리거 및 이벤트 바인딩부 순으로 수직적 흐름 구조화함.
AI 생성 코드 표기	AI가 생성한 코드 블록에 // AI-generated 주석 추가 - 실제 적용: 삼각함수 기하학 공식을 수동 렌더링 좌표로 변환하는 복잡한 수식 함수(drawRadar) 및 애니메이션 프레임 제어 이펙트(Ripple) 코드 블록 상단에 // AI-generated 헤더 주석을 의무 추가함.
기타	- 다중 오버레이 카드 UI 구조 하에서 부모 돔(DOM) 요소로의 강제 이벤트 전파 및 렌더링 무결성 파괴를 방지하기 위해, 하위 제어 노드(복사/외부 링크 이동 버튼 등) 클릭 핸들러에 event.stopPropagation() 적용을 의무 규정으로 수립함.

12. AI 활용 내역 요약

12-1. 팀 전체 AI 활용 현황

항목	내용
총 활용 횟수 (추정)	약 120회
주요 사용 도구	ChatGPT (GPT-4o), Gemini
가장 많이 활용한 단계	구현·검토 (M3)

12-2. 단계별 활용 내역

단계	주요 활용 내용	활용 도구	팀 수정 여부
기획 (M1)	타겟 사용자(대학생 및 일반 컴퓨터 비전문가)의 니즈 구체화를 위한 가상 페르소나 설계 및 업그레이드 시 잠재적 대기 불편 사항 도출	ChatGPT	수정 후 반영
설계 (M2)	단방향 연관 관계 구조 모델링 검토 및 Facade/Controller 패턴의 적절성에 대한 소프트웨어 아키텍처 피드백 확보	ChatGPT	수정 후 반영
구현·검토 (M3)	- 외부 라이브러리 없이 구동하는 동적 SVG 레이더 차트 좌표 계산식 생성 - 가격 크롤러 차단 방지용 requests 가짜 헤더 구성 및	ChatGPT, Gemini	수정 후 반영

	RAM 용량 파싱 정규표현식 보완		
--	-----------------------	--	--

12-3. AI 활용 3원칙 준수 자체 평가

원칙	준수 여부	비고
단순 복사 금지	준수	AI가 최초 제안한 복잡한 빌더(Builder) 설계 패턴 대신, 우리 프로젝트 규모에 어울리는 최적의 단일 상태 관리 구조(state 객체) 및 간결한 Facade(App 객체) 패턴으로 개발진이 리팩토링하여 적용함.
비판적 검증	준수	크롤링 실패 시 AI가 제시한 단순 터미널 에러 로깅 코드가 프론트엔드의 비정상 종료(Crash)를 야기함을 식별하고, 비동기 예외 시 더미 안전 데이터(Mock Data)를 대체 공급하는 무중단 보완 코드를 설계함.
수정 이력 명시	준수	리플(Ripple) 물결 애니메이션 함수 및 동적 차트 연산부(drawRadar) 등 인공지능이 제공한 특정 좌표 수학적 블록 영역 상단에 // AI-generated 주석 가이드라인을 철저하게 명시함.

12-4. 가장 효과적이었던 AI 활용 사례

- 대상 작업: 외부 라이브러리를 쓰지 않는 초경량 동적 SVG 레이더 차트(drawRadar) 좌표 연산식 개발
- 활용 방법:
 - 사용자의 진단 점수 데이터(CPU, GPU, RAM, 저장장치, 종합 점수)를 바탕으로

오각형의 정점 좌표를 동적으로 그려내기 위해 삼각함수 공식을 활용한 다각형 생성 수식을 질문했습니다.

- AI가 계산해 준 중심점 $\$cx, cy\$$ 대비 각 축의 라디안 점 도출 수식 공식인 $\$cx + d \cdot \cos(\theta)\$$ 및 $\$cy + d \cdot \sin(\theta)\$$ 변환 함수 코드를 바닐라 자바스크립트로 변환하여 App.drawRadar 내부 SVG 폴리곤 템플릿 리터럴로 직접 조립해 냈습니다.
- 수행 결과:
 - 고용량의 외부 차트 엔진(Chart.js 등)을 일절 로드할 필요가 없어 일렉트론 앱의 최초 구동 및 패키징 용량을 획기적으로 경량화했습니다.
 - 동시에 렌더링 리소스를 최소화하여 M3 최종 검증 단계에서 100% 무결하게 일치하는 반응형 결과 차트 뷰를 단 30라인의 스크립트만으로 완성해 냈습니다.

12-5. AI 활용의 한계 또는 주의가 필요했던 사례

- 대상 작업: 실시간 다나와 최저가 웹 크롤러 엔진(PriceSearcher) 셀렉터 파싱 및 자동화 개발
- 한계 극복 과정 및 교훈:
 - AI에게 타겟 하드웨어 쇼핑물의 HTML 원시 구조를 제공하고 최적의 크롤링 코드 작성을 요청했으나, AI는 대상 웹페이지가 자바스크립트에 의해 실시간 동적 로딩되는 동적 돔(DOM) 구조임을 완전히 간과한 채 Beautiful Soup의 단순한 정적 HTML 파서 코드를 제안했습니다. 이로 인해 최초 빌드 시 무조건 가격 필드가 None으로 반환되는 로직 결함이 드러났습니다.
 - AI의 제안을 무비판적으로 복사·붙여넣기 했다면 크롤러가 영구 동작하지 않았을 것입니다.
 - 개발팀은 직접 크롬 개발자 도구를 활용해 실시간 네트워크 비동기 API 요청 경로를 재분석하였고, 수집 시점에 가짜 브라우저 헤더(User-Agent)를 심지 않으면 타겟 서버에서 봇 트래픽으로 오인해 접근을 차단한다는 네트워크 보안 예외 사항을 직접 보완 구현했습니다. 이 과정을 통해 인공지능이 제시하는 코드는 반드시 개발팀의 실시간 도메인 디버깅과 검증을 필히 거쳐야만 상용 소프트웨어에 이식될 수 있다는 점을 뼈저리게 배웠습니다.

13. 회고 및 개선 사항

13-1. 팀 전체 회고

잘된 점

- 설계 명세의 실적 일치: M2 단계에서 조우찬 설계자가 수립한 클래스 다이어그램과 인터페이스 정의서가 매우 구체적이었기에, 전은호 개발자의 구현 단계(M3)에서 함수 선언부 불일치나 아키텍처 혼선 없이 통합 빌드가 원활하고 빠르게 완료되었습니다.
- 이슈 관리 기반의 일정 준수: 정승민 PM을 중심으로 깃허브 이슈(Issues)와 WBS 마일스톤을 실시간 동기화하여 지연 가능성이 큰 크롤링 모듈 개발 및 UI 연동 테스트를 한 주 앞당겨 검증 주간(13~14주차)을 넉넉하게 확보할 수 있었습니다.

아쉬운 점

- 외부 종속성에 따른 취약성: 최저가 실시간 검색을 담당하는 PriceSearcher 엔진이 타겟 외부 사이트의 HTML 돔(DOM) 태그 구조 및 보안 감지 차단(Anti-bot)에 높은 종속성을 가짐에 따라, 갑작스러운 레이아웃 변경 시 수집 예외 에러가 산발적으로 야기되었던 점이 안정성 면에서 아쉽습니다.
- 통합 화면 패키징 환경 조기 수립 부재: 일렉트론 데스크톱 런타임 환경과 파이썬 프로세스 간의 JSON 데이터 스트링 버퍼 수신 시점이 간헐적으로 지연되던 이슈를 구현 후반부에 발견하여, 상용 가공 환경에서의 디버깅 시간이 일부 집중되는 병목 현상이 있었습니다.

배운 점

- 설계 패턴과 품질의 비례관계: 단순 코드 작성보다 Facade 패턴 등의 적절한 디자인 구조가 실제 구현 코드의 가용성과 샌드박스 렌더러 보안(IPC 샌드박스)을 강화하고 디버깅 범위를 어떻게 단축하는지 기술적으로 실감했습니다.
- 비판적 시각을 동반한 AI 도구 제어: AI가 기계적으로 제안하는 고비용 설계 패턴(예: Builder 패턴 등)이 현재 개발 규모 대비 불필요한 복잡성을 유발할 수 있음을 깨닫고, 이를 비판적으로 필터링하여 최소한의 표준 규격 DTO 상태 객체로 단순화하는 주도적 제어 능력을 체득했습니다.

다음에 다시 한다면

- 실시간 HTML 웹 스크래핑 방식 대신 정형화되고 공식적인 오픈 마켓 제휴 API 연동을 우선 채택하거나, 로컬 DB 내에 캐시 레포지토리 테이블을 마련하여 타겟 쇼핑몰 응답 지연 상황에서도 유저 진단 결과 및 가상 대체 부품 정보가 무중단으로 표기되도록 구조적인 예외 극복 메커니즘을 전면 적용하겠습니다.

13-2. 팀원별 소감

이름	역할	한 줄 소감
정승민	PM	깃허브 이슈 트래킹을 통해 계획과 실제 실적 일정을 투명하게 조율하면서,

		주도적 일정 조율과 예산 관리의 중요성을 완벽하게 실감할 수 있는 값진 경험이었습니다.
하성권	분석가	비전문가 유저 관점에서 PC 사양 스캔을 가장 직관적으로 수집할 수 있는 요건을 탐색하고, 요구사항 명세가 설계 및 예외 처리 코드에 100% 매핑되는 과정을 확인하여 보람찼습니다.
조우찬	설계자	에러를 유발하던 Mermaid 다이어그램을 정석 클래스 구조로 재정립하고, SOLID 원칙이 실제 일렉트론 앱의 상태 관리 구조에 적용되는 아키텍처적 일치성에 깊은 성취감을 느꼈습니다.
전은호	개발자	일렉트론과 파이썬 비동기 통신 중 카드 링크의 이벤트 버블링 결함을 event.stopPropagation() 으로 원천 봉쇄하고 코딩 표준에 맞춰 완벽한 코드를 배포해 개발자로서 성장했습니다.
심금용	QA/보안	예산 수동 범위 제한 검증 및 API Key 은닉 보완 인스펙션을 주도하며, 상용 소프트웨어 배포 시 기능적 무결성 검증뿐 아니라 방어적 보안 검토가 왜 필수적인지 체험했습니다.